

Introduction

MeterFlow – Usage-Based API Billing Platform

Full Stack Implementation Guide

Project Overview

MeterFlow is a SaaS platform that allows developers to:

- Create APIs
- Generate API keys
- Track usage (per request)
- Apply rate limiting
- Calculate billing based on usage

This simulates real-world systems used by API companies and SaaS platforms.

Tech Stack

MERN Stack

- Frontend: React + Tailwind + React Query
 - Backend: Node.js + Express
 - Database: MongoDB
 - Realtime/Queue: Socket.io, BullMQ
 - Cache: Redis
-



Database Options (Use-case Based)

1 MongoDB (NoSQL)

- Best for: Flexible schema, logs, fast iteration
- Use for: API logs, usage tracking

👉 <https://www.mongodb.com/>

2 PostgreSQL (Relational)

- Best for: Billing, transactions, structured data
- Strong ACID compliance

👉 <https://www.postgresql.org/>

3 MySQL

- Best for: Traditional applications
- Easy hosting + wide support

👉 <https://www.mysql.com/>

4 Redis (In-Memory DB)

- Best for:
 - Rate limiting
 - Caching
 - Session storage

👉 <https://redis.io/>

Elasticsearch

- Best for:
 - Log searching
 - Analytics
 - Fast querying of large datasets

👉 <https://www.elastic.co/elasticsearch/>

Free Public APIs for Testing

Use these APIs to simulate usage billing:

1. Pokémon API

<https://pokeapi.co/>

2. JSON Placeholder (Fake REST API)

<https://jsonplaceholder.typicode.com/>

3. OpenWeather API

<https://openweathermap.org/api>

4. CoinGecko Crypto API

<https://www.coingecko.com/en/api>

5. Dummy Products API

<https://dummyjson.com/>



Advanced Features (Optional but Powerful)

- Webhooks for billing alerts
 - API analytics (latency, error rate)
 - Multi-tenant architecture
 - API playground
 - Audit logs
 - Role-based dashboards
-



Suggested Folder Structure (MERN)

backend/

- |— controllers/
- |— routes/
- |— middleware/
- |— models/
- |— services/
- |— utils/
- |— config/

frontend/

- |— components/
- |— pages/
- |— hooks/
- |— services/
- |— store/



Resume Description

Project: MeterFlow – API Billing Platform

- Built a scalable API gateway system to track and meter API usage
 - Implemented rate limiting, API key authentication, and request logging
 - Designed a billing engine for usage-based pricing
 - Developed real-time analytics dashboard using React
 - Integrated Stripe for subscription-based payments
-



Final Advice

Focus on:

- Gateway logic (this is your USP)
- Clean architecture
- Real-world features (billing, limits, analytics)

Avoid:

- Overcomplicating UI
 - Ignoring backend performance
-

Explanation of Project



Project Name (Portfolio Ready)

MeterFlow – Usage-Based API Billing & Metering Platform

Inspired by:

- Stripe Billing
 - RapidAPI
 - AWS API Gateway
 - OpenAI
-



What You Are Building (Simple Explanation)

A platform where developers can:

- Create APIs
 - Give API keys to users
 - Track how many requests each user makes
 - Charge users based on usage
-



Core System Architecture (High-Level)



Flow:

1. User signs up
2. Creates an API
3. Gets an unique API key
4. End users hit the API
5. System logs every request
6. Usage is tracked
7. Billing is calculated
8. User pays



CORE MODULES (Important)

1 Authentication & User System

- JWT + Refresh tokens
- Roles:
 - Admin
 - API Owner
 - Consumer

2 API Management System

- Create API
- Generate API keys
- Revoke keys
- Rotate keys (important 🔥)

3 API Gateway Layer (🔥 MOST IMPORTANT)

This is your **killer feature**.

Instead of calling APIs directly:

👉 All requests go through YOUR gateway

Responsibilities:

- Validate API key
 - Apply rate limiting
 - Log request
 - Forward request to actual API
-

4 Usage Tracking System

Store:

- API key
 - Endpoint
 - Timestamp
 - Response status
 - Latency
-

5 Billing Engine

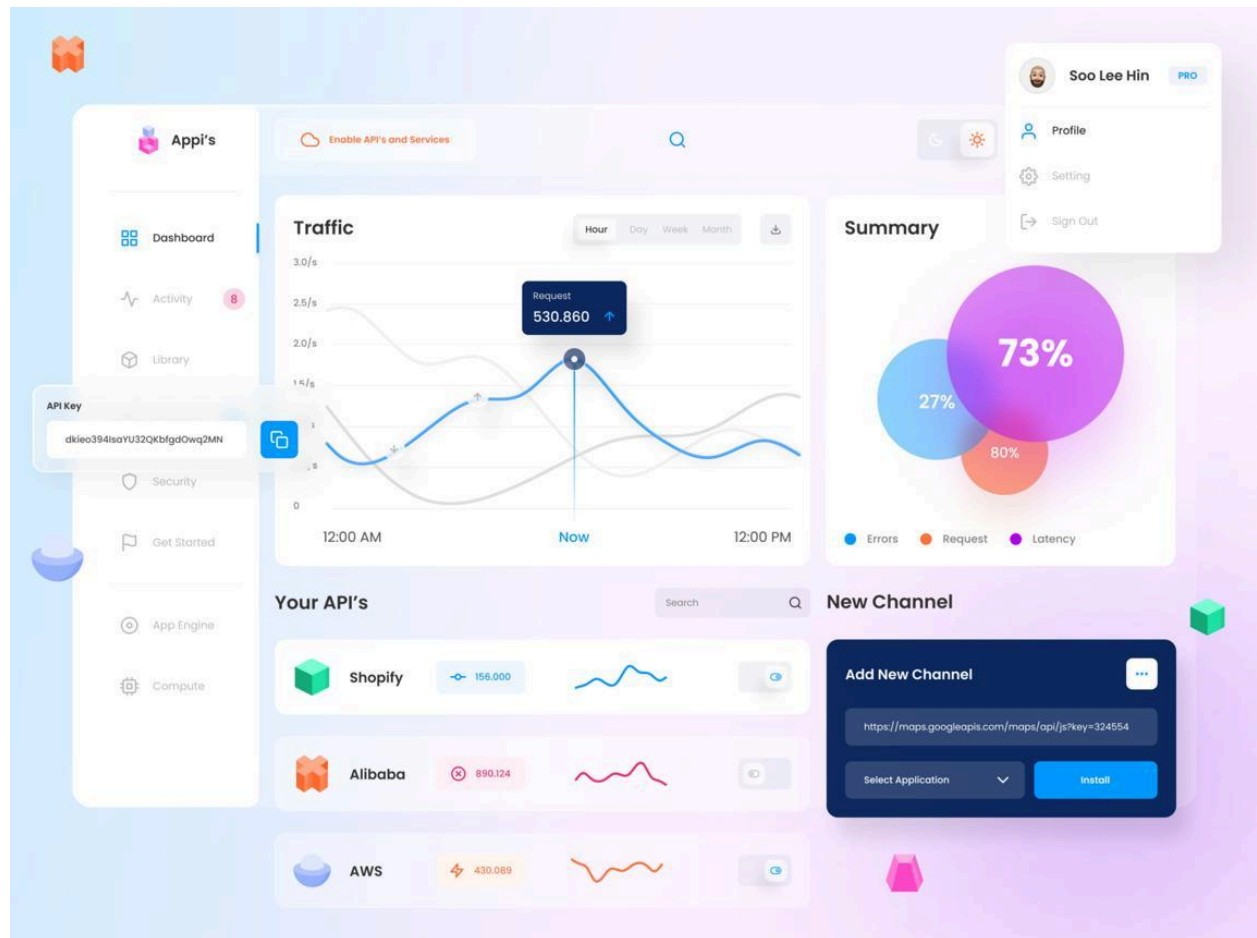
Pricing Example:

- Free: 1000 requests
- Pro: ₹0.5 / 100 requests

Logic:

- Count usage per user
 - Apply pricing
 - Generate invoice
-

6 Dashboard (Frontend)



CONSULT

Bookings

Team & Availability

Services

Clients

Settings

Company Profile

Manage Widgets

Payment Settings

Billing details

Notifications

Elsie Saunders

Billing details

Current subscription plan

\$25.00

Company Plus

Change Plan

Next payment

\$50.00

on May 15, 2020

Manage Payments

Payment history

Amount	Status	Receipient	Date	Payment method
\$65.00	Pending	Gabriel Banks	May 10, 2020	Visa 5432
\$250.00	Completed	Claudia Welch	Apr 10, 2020	Visa 5432
Billing plan Company Start 5 team members (\$8 / month each) + 100 GB extra storage (\$25.00) + 8 extra hours (\$2 per 1 hour) ID number EKG2SJFN Status Completed Invoice date Apr 10, 2020 Amount due \$250.00 Date paid Apr 10, 2020				
\$50.00	Completed	Nina Sherman	Mar 10, 2020	Visa 5432
\$50.00	Completed	Elizabeth Robbins	Feb 10, 2020	Visa 5432

Payment information

Mastercard 5432

Exp. date 12/24

Elsie Saunders

46 Mill Pond Dr.
New Rochelle, NY 10801

Storage

Using 0.65 GB from 1 GB included

Expand storage to get more space!

Buy Storage



7

Show:

- Total requests
- Active API keys
- Usage graph
- Errors
- Revenue

7 Payment Integration

- Stripe / Razorpay
- Subscription plans
- Payment history



Tech Stack (Strong Resume)

Frontend

- React
- Tailwind
- Chart.js / Recharts
- React Query

Backend

- Node.js
- Express
- MongoDB (with indexing 🔥)

Advanced

- Redis → Rate limiting
 - Nginx → Reverse proxy
 - Docker → Deployment
 - Queue (BullMQ) → Billing jobs
-



Database Design (Important for Interviews)

Users

- id, email, password, role

APIs

- id, userId, name, baseUrl

API Keys

- id, apid, key, status

Usage Logs

- id, apiKey, endpoint, timestamp, status

Billing

- userId, totalRequests, amount, status
-

Advanced Features (This Will Make You Stand Out)

Add these after MVP:

1. Rate Limiting

- Per API key
 - Per plan
-

2. API Analytics

- Requests per minute
 - Error rates
 - Latency
-

3. Webhooks

- Notify user when limit reached
-

4. API Playground

- Test API inside dashboard
-

5. Multi-Tenant Architecture

- Each company isolated
-



Freelancing Angle (VERY IMPORTANT)

You can sell this as:

“I build API monetization platforms for SaaS companies.”

Clients:

- AI startups
 - SaaS founders
 - Backend-heavy startups
-



How to Explain in Interview

Say:

“I built a usage-based billing system with an API gateway that intercepts and tracks every request, applies rate limiting, and calculates billing dynamically based on usage.”

That sounds like a **mid-level backend engineer**.



Pro Tip (Important)

Most people will:

✗ Just count requests

YOU should:

✓ Build a **gateway layer**

That's what makes this project elite.



2-WEEK EXECUTION PLAN



2-WEEK EXECUTION PLAN



Week 1 – Core Backend + Gateway

Day 1–2: Auth System

- JWT login/signup
 - Role system
 - API key generation
-

Day 3–4: API Management

- Create APIs
 - Generate/revoke keys
 - Store API metadata
-

Day 5–6: API Gateway (CORE)

- Middleware to:
 - Validate API key
 - Log request
 - Forward request
-

Day 7: Logging System

- Store:
 - API key
 - Endpoint
 - Timestamp
 - Response status
-



Week 2 – Billing + Dashboard + Deployment

Day 8–9: Usage Tracking + Billing Engine

- Count requests per user
 - Apply pricing logic
-

Day 10–11: Dashboard (Frontend)

- Usage charts
 - API key management
 - Billing summary
-

Day 12: Rate Limiting + Redis

- Limit requests per API key
-

Day 13: Payment Integration

- Stripe / Razorpay
 - Subscription plans
-

Day 14: Deployment

- Backend → Render / Railway
 - Frontend → Vercel
 - DB → Mongo Atlas
-

Resources

MERN Stack Resources (Node.js + React + MongoDB)

YouTube (Backend + System Concepts)

- Hitesh Choudhary – Node.js Backend Series
 - 👉 Covers JWT, APIs, middleware, production practices
 - Traversy Media
 - 👉 “Node.js API Authentication”, “REST API Crash Course”
 - Fireship
 - 👉 Short but powerful videos on system design, scaling, backend concepts
-

GitHub Repositories

- **Bulletproof Node.js Architecture**
<https://github.com/santig/bulletproof-nodejs>
 - 👉 Production-level folder structure (very important)
 - **Node.js API Boilerplate**
<https://github.com/hagopj13/node-express-boilerplate>
 - 👉 Auth, validation, structure ready
 - **Rate Limiting Example**
<https://github.com/nfriedly/express-rate-limit>
-

Documentation (Must Read)

- Express.js
 - 👉 <https://expressjs.com/>
 - Mongoose
 - 👉 <https://mongoosejs.com/docs/>
 - Redis
 - 👉 <https://redis.io/docs/>
 - Stripe
 - 👉 <https://stripe.com/docs>
-

Key Topics to Focus (MERN)

- Express Middleware (for API Gateway 🔥)
 - Rate Limiting with Redis
 - MongoDB Indexing (for logs)
 - Background Jobs (BullMQ)
-

API Data Sources (For Testing Your Platform)

Use these APIs to simulate real usage billing:

- PokeAPI
👉 <https://pokeapi.co/>
- JSONPlaceholder
👉 <https://jsonplaceholder.typicode.com/>
- OpenWeather
👉 <https://openweathermap.org/api>
- CoinGecko
👉 <https://www.coingecko.com/en/api>
- DummyJSON
👉 <https://dummyjson.com/>

Project Submission requirements

github link - backend github link & frontend github link
deploy - backend deploy link & frontend deploy link
demo video - screen record of frontend
documentation - read.md - with ss of frontend